



Sum-of-Max Chain Partition of a Tree

Ruixi Luo, Taikun Zhu, and Kai Jin^(✉)

School of Intelligent Systems Engineering, Shenzhen Campus of Sun Yat-Sen University, No. 66, Gongchang Road, Shenzhen 518107, Guangdong, People's Republic of China
jink8@mail.sysu.edu.cn

Abstract. Path partition problems on trees have found various applications. In this paper, we present an $O(n \log n)$ time algorithm for solving the following variant of path partition problem: given a rooted tree of n nodes $1, \dots, n$, where vertex i is associated with a weight w_i and a cost s_i , partition the tree into several disjoint chains $C_1, \dots, C_k$, so that the weight of each chain is no more than a threshold w_0 and the sum of the largest s_i in each chain is minimized. We also generalize the algorithm to the case where the cost of a chain is determined by the s_i of the vertex with the highest rank in the chain, which can be determined by an arbitrary total order defined on all nodes instead of the value of s_i .

Keywords: Chain Partition · Tree Partition · Binomial Heap · Dynamic Programming · Heap-over-Heap

1 Introduction

Sequence and tree partition problems have been extensively studied in the past decades [2, 3, 5, 23]. In this paper, we study a variant of the tree partition problems, namely, the sum-of-max chain partition of a tree.

Above all, it is necessary to clarify the definition of a chain in the context of tree partitioning. We adopt the chain partition definition as in Misra and Tarjan's work [21]. A chain in this context is defined as follows: A path of a rooted tree is a *chain* if for each node pair (u, v) on the path, either u is the ancestor of v , or v is the ancestor of u .

Sum-of-max chain partition problem. Given a rooted tree T of n nodes $1, \dots, n$, where each vertex i is associated with a weight w_i and a cost s_i , partition the tree into several disjoint chains $C_1, \dots, C_k$, where k is arbitrary, so that the total weight of each chain is no more than a given threshold w_0 and the sum of the largest s_i in each chain is minimized. Without loss of generality, we assume that $w_i \leq w_0$ for all i .

This research is supported by Department of Science and Technology of Guangdong Province (Project No. 2021QN02X239) and Shenzhen Science and Technology Program (Grant No. 202206193000001, 20220817175048002).

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
F. V. Fomin and M. Xiao (Eds.): COCOON 2025, LNCS 15984, pp. 53–67, 2026.
https://doi.org/10.1007/978-981-95-0218-9_5

Let the largest s_i in each chain be referred to as the **cost** of the chain. The total cost of a chain partition $P = \bigcup_{i=1}^k C_i$ is the sum of the largest parameter s in each chain, as defined above. Formally, $\text{cost}(P) = \sum_{1 \leq i \leq k} \max_{j \in C_i} s_j$.

Partitioning a tree into paths has been studied by a number of researchers. In [27], the problem of partitioning a tree into paths with each path having no more than k nodes was solved with a linear time algorithm. The sum-of-max chain partition problem on trees can be seen as a generalization of the problem with variable weights and costs on nodes to some extent. The main contribution of this paper is an $O(n \log n)$ time algorithm for solving sum-of-max chain partition problem on trees mentioned above.

1.1 Related Works

Misra and Tarjan [21] investigated a type of chain partition problem where a tree is divided into several chains, each of which does not exceed the weight limit w_0 , and the objective is to minimize the cost of edges contained in no chains. An $O(n \log n)$ algorithm for this problem was presented in their paper.

The *heavy-light decomposition* technique [25] introduces a type of chain partition of trees, which finds numerous applications [1, 7, 11, 15]. Similarly, the *long-path decomposition* technique [4] defines another chain partition of trees, which can be used to speed up several dynamic programming algorithms on trees. Both of the mentioned chain partitions can be computed in linear time.

Path partition problems have drawn more attention in the literature [6, 9, 16, 22]. For example, partitioning a graph G into the minimum number of paths has been studied extensively [5, 17, 19]. Boesch et al. [5] provided a simple polynomial algorithm for the special case where G is a tree. This type of partition problem on trees is related to the Hamiltonian completion of a tree [10, 24].

Some path partition problems of trees with costs related to the number of paths have also been studied in the past. Cai et al. [8] studied ω -path partition of a edge-weighted tree and gave a linear time algorithm for the problem. Jin and Li [14] presented a linear path partition algorithm for edge-colored trees. Yan et al. [27] presented a linear-time algorithm for the k -path partition problem in trees, which considers partitioning a tree into the smallest number of paths such that each path has at most k vertices. The path partitions problems mentioned above all aim for minimizing the number of the resulting paths. This kind of objective can be seen as a special case of sum-of-max objective with uniform costs for all elements. In other words, by setting uniform costs for all elements, the sum-of-max objective degenerates into minimizing the number of paths in the partition. As a sequence can be treated as a tree without branches, the problem in this paper can be seen as an extension of the sum-of-max sequence partition problem studied in [13], where an $O(n)$ time algorithm is given.

Partition a tree into subtrees has also been investigated extensively. A few references about such problems can be found in the introduction of [13].

The decision version of our chain partition problem is NP-complete for a directed graph G . This is because the chain partition problem can be reduced to the minimum path cover problem when all weights and costs are set to be 1.

Then, the minimum path cover problem can further be reduced to a classic NP-complete problem, namely the Hamiltonian path problem [12], since a minimum path cover consists of one path if and only if there is a Hamiltonian path in G .

1.2 Applications

The partition problem on trees can be applied in real-life, such as water environment protection, industrial investment planning, power grid maintenance, biological sample supply and preservation, etc.

In a river environmental protection scenario, we need to send several staff members to clean the river. Each staff member is responsible for a continuous section of the river. Since the river forks, each unbranched river segment can be seen as a vertex of the tree.

Assume that each person can clean a river of length w_0 at most, each segment i is associated with length w_i and equipment requirement s_i . The equipment that each staff member needs is determined by the largest s_i of the river segments he is responsible for. An effective sum-of-max chain partition guarantees a good staff allocation plan with minimum total equipment requirement.

Our sum-of-max chain partition problem can also be applied to industrial investment. Assume a cooperation focuses on oil related industries. Then, the supply chains based on oil can be seen as a tree, with oil as the root, various consumer goods as leaf vertices and intermediate products between them. Now, the cooperation intends to establish a number of factories, each produces several products on the tree.

In order to ensure the continuity of production, only continuous products on the tree, i.e. a chain, can serve as the supply chain segment for a factory's production. Suppose a product i on the tree requires w_i area of land, and s_i level of pollution. Each factory can occupy no more than w_0 area of land and its pollution is determined by the largest s_i of the products it produces. A good sum-of-max chain partition can reduce the total pollution of the factories.

2 Preliminaries

Denote the given tree and its subtree rooted at vertex v by T and T_v , respectively. For each vertex u in T_v , denote by $T[v, u]$ the chain from v to u , by $T[v, u)$ the chain from v to the parent of u , and by $T(v, u]$ the chain $T[v, u]$ without v . By this definition, $T[v, v)$ is empty. The set of children of v is denoted by $\text{Child}(v)$.

Definition 1. The window of v , denoted by win_v , refers to the set of vertices u in T_v for which the chain $T[v, u]$ weights no more than w_0 .

A vertex u in T_v is s -maximal if each vertex p in $T[v, u)$ admits that $s_p < s_u$. For $u \in T_v$ and $u \neq v$, let $\text{next}_v(u)$ be the closest proper s -maximal ancestor of u in T_v . Note that $\text{next}_v(u)$ is well-defined since the root v of T_v is always s -maximal in T_v .

Let win_v^* denote the s -maximal vertices in win_v and win_v^- denote vertices in $\text{win}_v \setminus \text{win}_v^*$.

Our algorithm follows a dynamic programming paradigm.

Let $F[v]$ denote the minimum cost to partition T_v into chains. Obviously, our goal is to compute $F[r]$, where r denotes the root of T .

Let us consider how to find the optimal partition of T_v . Obviously, v must belong to some chain, e.g. $T[v, i]$. Define $\text{cost}(v, i)$ to be the minimum cost to partition T_v into chains, one of which equals $T[v, i]$. Then, we have

$$F[v] = \min_{T[v, i] \text{ weights no more than } w_0} \text{cost}(v, i). \quad (1)$$

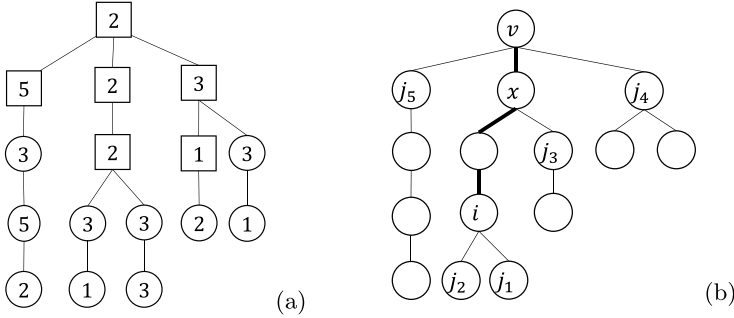


Fig. 1. (a) Illustration for window and outer boundary, where $w_0 = 7$. The numbers in the vertices are their weights. Squares represent the vertices in the window. (b) Illustration for $\text{sum}(v, i)$. Here, $\text{sum}(v, i) = F[j_1] + F[j_2] + F[j_3] + F[j_4] + F[j_5]$.

Recall Definition 1. We rewrite Eq. (1) as:

$$F[v] = \min_{i \in \text{win}_v} \text{cost}(v, i). \quad (2)$$

Recall next in Definition 1. A trivial formula of $\text{cost}(v, i)$ is as follows:

$$\text{cost}(v, i) = \begin{cases} \text{sum}(v, i) + s_i & \text{if } i \in \text{win}_v^* \\ \text{sum}(v, i) + s_{\text{next}_v(i)} & \text{if } i \in \text{win}_v^-, \end{cases} \quad (3)$$

where

$$\text{sum}(v, i) := \sum_{j's \text{ parent} \in T[v, i], j \notin T[v, i]} F[j]. \quad (4)$$

See Fig. 1 (b) for an illustration of $\text{sum}(v, i)$.

By the definitions above, we can derive the following lemma, which is crucial for the design and analysis in this paper.

Lemma 1. Assume $i \in \text{win}_v^-$ and $x = \text{next}_v(i)$. Then,

$$i \in \text{win}_x^- \text{ and } x = \text{next}_x(i), \quad (5)$$

and more importantly,

$$\text{cost}(v, i) - \text{cost}(x, i) = \text{cost}(v, x) - \text{cost}(x, x). \quad (6)$$

Proof. We may assume $v \neq x$ (otherwise it is trivial).

Since $i \in \text{win}_v^-$ and $x = \text{next}_v(i)$, we know that $s_x \geq s_p$ for $p \in T[v, i]$. In particular, $s_x \geq s_p$ for $p \in T[x, i]$. Consequently, in $T[x, i]$, only x is s-maximal in T_x and thus we obtain Eq. (5).

Further applying Eq. (3), we get

$$\begin{cases} \text{cost}(v, i) = \text{sum}(v, i) + s_x \\ \text{cost}(x, i) = \text{sum}(x, i) + s_x. \end{cases}$$

Let p_x denote the parent of x .

$$\begin{aligned} \text{cost}(v, i) - \text{cost}(x, i) &= (\text{sum}(v, i) + s_x) - (\text{sum}(x, i) + s_x) \\ &= \text{sum}(v, i) - \text{sum}(x, i) \\ &= \text{sum}(v, p_x) - F[x]. \end{aligned} \tag{7}$$

The last equation follows from the definition (4).

Similarly, we have

$$\begin{aligned} \text{cost}(v, x) - \text{cost}(x, x) &= (\text{sum}(v, x) + s_x) - (\text{sum}(x, x) + s_x) \\ &= \text{sum}(v, x) - \text{sum}(x, x) \\ &= \text{sum}(v, p_x) - F[x]. \end{aligned} \tag{8}$$

Equations (7) and (8) together imply Eq. (6). $\square$

We will use a few heaps to compute the minimum cost $F[v]$. For simplicity and time complexity, all heaps in this paper refer to *binomial heaps* [26]. (The *leftist tree* [20] is not appropriate for our purpose.)

Table 1. Operations for the heaps.

Operation	Definition	Time required
find-min(max)(H)	finds the minimal(maximal) element in heap H .	$O(1)$
insert(H, i, k)	inserts vertex i in heap H with key k .	$O(\log n)$
delete-vertex(H, i)	deletes vertex i in heap H (if exists).	$O(\log n)$
meld(H_1, H_2)	melds heap H_1 with H_2 into H_1 .	$O(\log n)$
add-all(H, v)	adds value v to all key values in H .	$O(\log n)$
key-value(H, i)	returns key value of i in H (if exists).	$O(\log n)$

Operations shown in Table 1 are needed for the binomial heaps in the algorithm. Most of the operations are originally available for the binomial heaps [26]. Additionally, we implement a lazy tag technique to create an add-all() operation in Table 1, which is discussed in the following subsection.

2.1 Binomial Heap with Lazy Tags

For each node in the binomial heap, we add a lazy tag to it, representing adding or subtracting a value uniformly for all elements below. In this section, we briefly explain the differences brought by the lazy tag technique.

Traditionally, when using lazy tags, we pushdown a tag to all children when the node is accessed. However, this would lead to a non- $O(1)$ pushdown operation in binomial heaps since there can be $O(\log n)$ children for one node, which may cause an $O(\log^2 n)$ factor in time bound.

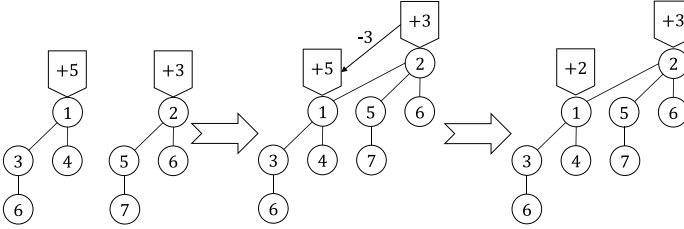


Fig. 2. Example of binomial heaps melding with lazy tag

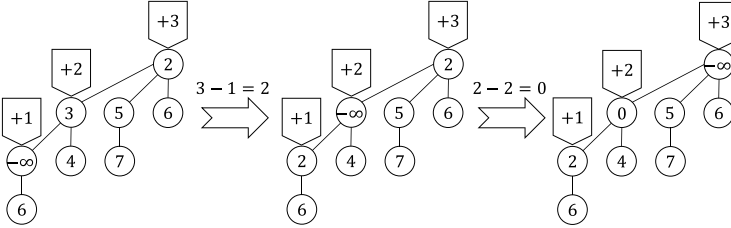


Fig. 3. Example of binomial heaps pushing up a node $-\infty$. In the first step, 3 is swapped down and becomes $3 - 1 = 2$ to counter the $+1$ tag's effect since it was not affected by $+1$ in the first place. Similarly, 2 is swapped down and becomes $2 - 2 = 0$ to counter the $+2$ tag.

Therefore, we do not pushdown a lazy tag unless it's a tag of a root vertex to be deleted. This change requires us to handle the tags in a slightly different way. First, we only exert a negative pushdown to the newly melded tree as shown in Fig. 2. Suppose tree B_a with a bigger root is linked to B_b , then the opposite number of the lazy tag of B_b will be added to the lazy tag of B_a during the linking operation. This trick allows us to preserve the property of the heap while the negative pushdown is still an $O(1)$ operation.

Besides, when pushing up a node for a key value change. The key value of the node swapped down needs to be adjusted since the tag stays still. If a vertex is swapped down, it needs to counter the effect caused by the lower tag and thus add the opposite of the lower tag, see Fig. 3 for illustration.

Thanks to the $O(\log n)$ depth limit of binomial heap, even with lazy tags affecting the real key value of each node, we can still also acquire a node's key-value in $O(\log n)$ time by simple tree-climbing towards the root.

3 Algorithm for Finding the Optimal Chain Partition

Recall Eq. (2). This section shows how we compute $F[v]$ efficiently.

For computing $F[v]$, we use an interesting combined data structure, which we call *heap-over-heap*. Briefly, there are several disjoint heaps in the first layer (see $H_x^{(v)}$'s below), and the minimum elements of all heaps in the first layer are maintained in another heap (see $\mathbb{H}^{(v)}$ below) at the second layer.

Min-heap $H_x^{(v)}$ for $x \in \text{win}_v^*$.

$H_x^{(v)}$ stores i with key $\text{cost}(x, i)$, if $(i = x)$ or $(i \in \text{win}_v^- \text{ and } \text{next}_v(i) = x)$.

Min-heap $\mathbb{H}^{(v)}$.

$\mathbb{H}^{(v)}$ stores $i = \text{find-min}(H_x^{(v)})$ with key $\text{cost}(v, i)$, for each $x \in \text{win}_v^*$.

Each $i \in \text{win}_v$ belongs to exactly one heap in $\{H_x^{(v)} : x \in \text{win}_v^*\}$. See Fig. 4.

Remark 1. Be aware that i in $H_x^{(v)}$ is associated with a key value $\text{cost}(x, i)$ instead of $\text{cost}(v, i)$. This is crucial to the entire algorithm. If i is associated with $\text{cost}(v, i)$ in the first layer, it is not easy to update the values when v is changed.

Lemma 2. $\arg \min_{i \in \text{win}_v} \text{cost}(v, i) = \text{find-min}(\mathbb{H}^{(v)})$. In other words, computing $F[v] = \min_{i \in \text{win}_v} \text{cost}(v, i)$ reduces to computing the top element of $\mathbb{H}^{(v)}$.

Proof. Take an arbitrary $i \in \text{win}_v$. Assume i is in $H_x^{(v)}$.

Let $i' = \text{find-min}(H_x^{(v)})$. We have $\text{cost}(x, i) \geq \text{cost}(x, i')$.

Since i, i' are in $H_x^{(v)}$, respectively, applying Lemma 1,

$$\begin{aligned} \text{cost}(v, i) - \text{cost}(x, i) &= \text{cost}(v, x) - \text{cost}(x, x) \\ \text{cost}(v, i') - \text{cost}(x, i') &= \text{cost}(v, x) - \text{cost}(x, x). \end{aligned}$$

Therefore, $\text{cost}(v, i) - \text{cost}(v, i') = \text{cost}(x, i) - \text{cost}(x, i') \geq 0$, namely, $\text{cost}(v, i) \geq \text{cost}(v, i')$. Since $a = \text{find-min}(\mathbb{H}^{(v)})$ and i' is also in $\mathbb{H}^{(v)}$, we have $\text{cost}(v, i') \geq \text{cost}(v, a)$. Thus for any $i \in \text{win}_v$, $\text{cost}(v, i) \geq \text{cost}(v, a)$.

Further since $a \in \text{win}_v$ (trivial), we have $a = \arg \min_{i \in \text{win}_v} \text{cost}(v, i)$. $\square$

For maintaining the heap $\mathbb{H}^{(v)}$ over heaps $H_x^{(v)}$'s structure efficiently, we have to maintain two more types of heaps: (1) A heap (see $W^{(v)}$ below), which is used for detecting the vertices leaving the window win_v . (2) A bunch of heaps (see $S_x^{(v)}$'s below), which are used for organizing the s-maximal vertices.

Max-heap $W^{(v)}$.

$W^{(v)}$ stores i with key $\sum_{j \in T[v,i]} w_j$, for each $i \in \text{win}_v$.

Min-heap $S_x^{(v)}$ for $x \in \text{win}_v^*$.

$S_x^{(v)}$ stores i with key s_i , for each $i \in \text{win}_v^*$ with $\text{next}_v(i) = x$. See Fig. 4.

Remark 2. In our algorithm, for each x , we do not have to store all the $H_x^{(v)}$'s one by one for each v with $x \in \text{win}_v^*$. Instead, all such $H_x^{(v)}$'s can share one entity in the memory, denoted as H_x . The notation $H_x^{(v)}$ can be viewed as a historical version of H_x at the stage of computing v . The same logic holds for $S_x^{(v)}$.

3.1 Maintain the Heaps Efficiently

We compute $\{F[v]\}$ from bottom to up. Suppose $F[v]$ is to be computed in the algorithm. It means that we have already computed the $F[c]$ for all $c \in \text{Child}(v)$ and built all the aforementioned heaps related to T_c . Our current task is to obtain the heaps related to T_v from those heaps built for T_c 's.

Consider an example as shown in Fig. 5. At the stage for computing $F[v]$,

1. vertices f, g, h are no longer in the window (namely, $f, g, h \in \text{win}_c \setminus \text{win}_v$), and hence should be eliminated from the heaps containing them;
2. vertices c, d are no longer s-maximal (as $s_v \geq s_c$ and $s_v \geq s_d$), and hence $\{S_x^{(v)} : x \in \text{win}_v^*\}$ changes accordingly; this leads to some merges of heaps in $\{H_x^{(v)} : x \in \text{win}_v^*\}$, and furthermore, $\mathbb{H}^{(v)}$ should be updated accordingly.

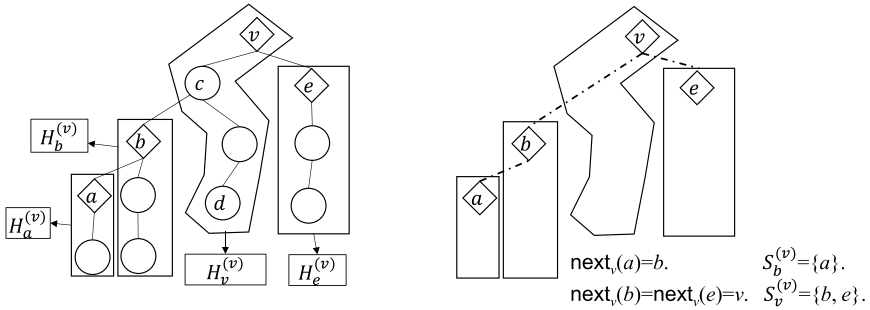


Fig. 4. A partition of win_v into different parts, with each part organized by a heap. In this figure, rhombuses indicate s-maximal vertices and circles indicate other vertices.

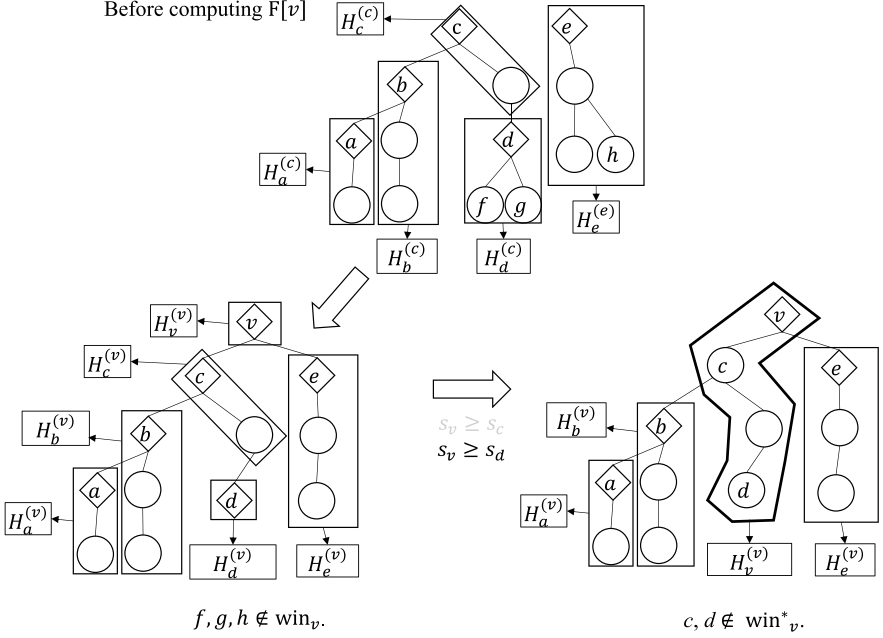


Fig. 5. Illustration of the change of the heaps at the stage for computing $F[v]$.

From the example, we see two major procedures are required at the moment:

- (a) Find $i \in \text{win}_c \setminus \text{win}_v$. Then, eliminate them from corresponding heaps.
- (b) Find $x \in \text{win}_c^* \setminus \text{win}_v^*$ (it loses s-maximality due to $s_x \leq s_v$), and find those i for which $\text{next}_c(i) = x$ (due to $\text{next}_v(i) = v \neq x$) (by utilizing $S_x^{(v)}$). Then, modify $\{S_x^{(v)} : x \in \text{win}_v^*\}$ and the heap-over-heap structure accordingly.

We present the main process of our algorithm as Algorithm 1 below.

In the algorithm, we first add v into $H_v^{(v)}$ with key $s_v + \sum_{c \in \text{Child}(v)} F[c]$ for initialization of v . Then, for every $c \in \text{Child}(v)$, we do the following:

- First, add c to $S_v^{(v)}$ as part of the initialization of $S_v^{(v)}$, since c is s-maximal by default in T_c . We assume for the time being c is still s-maximal in T_v before we check s_i in the following process. Therefore, c first goes to $S_v^{(v)}$.
- We then add w_v to all keys in $W^{(v)}$, so that the vertices in $W^{(v)}$ with key greater than w_0 are vertices in $\text{win}_c \setminus \text{win}_v$ which need to be deleted (recall procedure (a)). We call a function $\text{Over-weight}(c)$ to delete the corresponding vertices from all heaps; see appendix B.1 of the full paper [18] for details. After $\text{Over-weight}(c)$, we meld $W^{(c)}$ into $W^{(v)}$.
- Next, we meld $\mathbb{H}^{(c)}$ into $\mathbb{H}^{(v)}$ as a part of the initialization of $\mathbb{H}^{(v)}$. By Lemma 1, since c is considered to be s-maximal in T_c by default, we add $(\sum_{c \in \text{Child}(v)} F[c]) - F[i]$ to all vertices in $\mathbb{H}^{(c)}$ to update the $\text{sum}(c, *)$ of every

for $c \in \text{Child}(v)$ do $F[c] \leftarrow \text{compute_}F(c);$ $a \leftarrow \sum_{c \in \text{Child}(v)} F[c];$ $H_v^{(v)}, \mathbb{H}^{(v)}, S_v^{(v)}, W^{(v)} \leftarrow \emptyset;$ insert($H_v^{(v)}, v, s_v + a$); for $c \in \text{Child}(v)$ do insert($S_v^{(v)}, c, s_c$); add-all($W^{(v)}, w_v$); Over-weight(c); meld($W^{(v)}, W^{(v)}$); $b \leftarrow a - F[c];$ add-all($\mathbb{H}^{(c)}, b$); meld($\mathbb{H}^{(v)}, \mathbb{H}^{(c)}$); Update-s-maximal(v); insert($W^{(v)}, v, w_v$); insert($\mathbb{H}^{(v)}, \text{find-min}(H_v^{(v)}), \text{key-value}(H_v^{(v)}, \text{find-min}(H_v^{(v)}))$); return key-value($\mathbb{H}^{(v)}, \text{find-min}(\mathbb{H}^{(v)})$);	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
--	---

Algorithm 1: compute_ $F(v)$

key to $\text{sum}(v, *)$ before melding. As for updating the s_i part of the costs, we leave this issue to function $\text{Update-s-maximal}(v)$ later.

After the above process for all $c \in \text{Child}(v)$, heap $S_v^{(v)}$ contains all $\text{Child}(v) \cap \text{win}_v$. $W^{(v)}$ and all H 's only contain vertices in win_v , which are the only vertices that we need for the heaps of T_v .

We now focus on the whole T_v . Here, we consider the vertices that lose its s-maximal identity after adding v as the new root (as mentioned in procedure (b)). We process these vertices in $\text{Update-s-maximal}(v)$ with consecutive updating and melding (For more detailed description of functions $\text{Over-weight}()$ and $\text{Update-s-maximal}()$, see the full version of this paper [18].)

After $\text{Update-s-maximal}(v)$, we add v into $W^{(v)}$ with key w_v for further recursive calls and insert the top vertex of new $H_v^{(v)}$ into $\mathbb{H}^{(v)}$. Then, we have all heaps meeting their definitions in T_v (see appendix C of the full version of the paper [18]), and all we need is to extract minimum choice from $\mathbb{H}^{(v)}$ to obtain $F[v]$.

The reader can refer to Algorithm 2 and 3 by the end of Subsect. 3.2 for the pseudocode of $\text{Over-weight}()$ and $\text{Update-s-maximal}()$.

3.2 Analysis

Lemma 3. $\text{Over-weight}()$ takes $O(n \log n)$ time in total.

Lemma 4. $\text{Update-s-maximal}()$ takes $O(n \log n)$ time in total.

Lemmas above are proved in the full version of this paper (appendix B) [18].

Theorem 1. *The sum-of-max chain partition problem can be solved in $O(n \log n)$ time with our algorithm.*

Proof. By Lemma 3 and Lemma 4, the functions Over-weight() and Update-s-maximal() require $O(n \log n)$ time in total.

In the main process of each v , we insert v into $H_v^{(v)}$ with insert() in $O(\log n)$ time. Note that the acquisitions of $a = \sum_{c \in \text{Child}(v)} F[c]$ in total take time

$$O((\sum_{v \in T} |\text{Child}(v)|) \log n) = O(n \log n).$$

Before Update-s-maximal(v), for each $c \in \text{Child}(v)$, we insert c into $S_v^{(v)}$ with insert() in $O(\log n)$ time. We then update $W^{(v)}$ in $O(\log n)$ time by add-all(). After Over-weight(c), we also update and meld $\mathbb{H}^{(c)}$ into $\mathbb{H}^{(v)}$ with add-all() and meld() in $O(\log n)$ time. Here, we conduct $O(1)$ times of $O(\log n)$ operations (excluding Over-weight(c)) for every $c \in \text{Child}(v)$ as described above. These operations in total also take time

$$O((\sum_{v \in T} |\text{Child}(v)|) \log n) = O(n \log n).$$

After Update-s-maximal(v), we insert v into $W^{(v)}$ and insert the top vertex of the new $H_v^{(v)}$ into $\mathbb{H}^{(v)}$ with insert() in $O(\log n)$ time. The extraction of $F[v]$ is also $O(\log n)$ with find-min() and key-value(). Here, we conduct $O(1)$ times of $O(\log n)$ operations (excluding Update-s-maximal(v)) for v , which is also $O(n \log n)$ in total.

Therefore, during the whole recursive process, the algorithm takes $O(n \log n)$ time. $\square$

<pre> while key-value($W^{(v)}$, find-max($W^{(v)}$)) > w_0 do $x \leftarrow$ find-max($W^{(v)}$); delete-vertex($S_{\text{next}_c(x)}^{(v)}$, x); if find-min($\mathbb{H}^{(c)}$) = x then $g \leftarrow$ key-value($\mathbb{H}^{(c)}$, x) - key-value($H_{\text{next}_c(x)}^{(v)}$, x); delete-vertex($H_{\text{next}_c(x)}^{(v)}$, x); $k \leftarrow$ find-min($H_{\text{next}_c(x)}^{(v)}$); insert($\mathbb{H}^{(c)}$, k, key-value($H_{\text{next}_c(x)}^{(v)}$, k) + g); delete-vertex($\mathbb{H}^{(c)}$, x); else delete-vertex($H_{\text{next}_c(x)}^{(v)}$, x); delete-vertex($W^{(v)}$, x); </pre>	<pre> 1 2 3 4 5 6 7 8 9 10 11 12 </pre>
---	---

Algorithm 2: Function Over-weight(c).

while key-value($S_v^{(v)}$, find-min($S_v^{(v)}$)) $\leq s_v$ do	1
$x \leftarrow \text{find-min}(S_v^{(v)});$	2
$z \leftarrow \text{find-min}(H_x^{(v)});$	3
$g \leftarrow \text{key-value}(\mathbb{H}^{(v)}, z) - \text{key-value}(H_x^{(v)}, z) + s_v - s_x;$	4
add-all($H_x^{(v)}, g$);	5
meld($H_v^{(v)}, H_x^{(v)}$);	6
delete-vertex($\mathbb{H}^{(v)}, z$);	7
meld($S_v^{(v)}, S_x^{(v)}$);	8
delete-vertex($S_v^{(v)}, x$);	9

Algorithm 3: Update-s-maximal(v)

Remark 3. The maintenance and acquisition of $\text{next}_*(\cdot)$ are $O(n\alpha(n)) < O(n \log n)$ in total. Therefore, function $\text{next}_*(\cdot)$ does not affect the $O(n \log n)$ time bound.

The acquisition of $\text{next}_*(\cdot)$ can be done in $O(\alpha(n)) < O(\log n)$ time with the $\text{find}(\cdot)$ operation of disjoint-set data structures. To achieve that, all we need to do is to set every vertex $\text{next}_c(c)$ of $c \in \text{Child}(v)$ as v when $F[v]$ starts to be computed and merge the sets of vertex $x \in T_v$ and v when x loses its s-maximal identity in $\text{Update-s-maximal}(v)$.

As a result, the $\text{find}(\cdot)$ operation can be seen as $\text{next}_c(\cdot)$ before $\text{Update-s-maximal}(v)$ and $\text{next}_v(\cdot)$ after $\text{Update-s-maximal}(v)$ by the definition of next . Thus we have $\text{next}_c(\cdot)$ and $\text{next}_v(\cdot)$ ready for every T_c of $c \in \text{Child}(v)$ in the main process of computing $F[v]$.

4 A Generalized Chain Partition Problem

This section discusses a generalization of our algorithm.

Suppose every vertex a of the tree is associated with a distinct rank r_a . The *dominant vertex* of a chain refers to the one in the chain with the largest rank.

A chain partition problem. Given a rooted tree T of n nodes, where vertex i is associated with a weight w_i , a cost s_i , and a rank r_i . Partition the tree into several disjoint chains $C_1, \dots, C_k$, where k is arbitrary, so that the total weight of each chain is no more than a given threshold w_0 and the sum of the costs of the chains is minimized, where the cost of a chain is given by s_i – where i denotes the dominant vertex of the chain.

This problem contains the previous problem as a special case; by setting $r_i = s_i$ we obtain the original sum-of-max problem.

To tackle this problem, we replace the concept s-maximal with r-maximal, and this also leads to a change of the definition of $\text{next}_v()$ and win^* :

Definition 2. A vertex u in T_v is r-maximal if each vertex p in $T[v, u)$ admits that $r_p < r_u$. For $u \in T_v$ and $u \neq v$, let $\text{next}_v(u)$ be the closest r-maximal proper ancestor of u in T_v . Let win_v^* denote the r-maximal vertices in win_v and win_v^- denote the vertices in $\text{win}_v \setminus \text{win}_v^*$.

We now slightly modify our algorithm to solve the generalized problem:

One thing we have to change lies in the definition of heaps: $S_x^{(v)}$'s keys should be the r_i 's of vertices instead of s_i 's. Another change we make lies in the function $\text{Update-s-maximal}()$ in appendix B.2 of the full version of the paper [18]. All we need to do is to change the criteria from s-maximal to r-maximal (by changing the condition $\leq s_v$ to $\leq r_v$).

The definitions above still fit Lemma 2, Eq. (2) and (3) and all relevant analysis in this paper. The heaps can be maintained and implemented to compute $F[v]$'s using the algorithm described above. As a summary, we conclude that

Theorem 2. The chain partition problem above can be solved in $O(n \log n)$ time.

5 Conclusions

In this paper, we present an $O(n \log n)$ algorithm for the sum-of-max chain partition of a tree under a knapsack constraint. The algorithm is based on non-trivial observations (such as Lemmas 1 and 2). The key ingredient of our algorithm is a delicately designed data structure that consists of two layers of heaps, where the elements of heaps in the second layer $\mathbb{H}^{(v)}$ come from the minimum elements of the heaps in the first layer $H^{(v)}$'s. We also generalize the algorithm to handle a more complex scenario (Sect. 4), and the time complexity remains $O(n \log n)$.

One question worth exploring in the future is how to find the sum-of-max path partition of a tree. Moreover, can we find the optimal chain partition of a tree when the cost of a chain C is defined as $\max_{i \in C} s_i - \min_{i \in C} s_i$?

References

1. Alstrup, S., Lauridsen, P.W., Sommerlund, P., Thorup, M.: Finding cores of limited length. In: Dehne, F., Rau-Chaplin, A., Sack, J.-R., Tamassia, R. (eds.) WADS 1997. LNCS, vol. 1272, pp. 45–54. Springer, Heidelberg (1997). https://doi.org/10.1007/3-540-63307-3_47
2. Bagheri, A., Keshavarz-Kohjerdi, F., Razzazi, M.: A linear-time approximation algorithm for the minimum-length geometric embedding of trees. Optimization 1–16 (2024)

3. Bagheri, A., Razzazi, M.: Minimum height path partitioning of trees. *Sci. Iranica* **17**(2) (2010)
4. Bender, M., Farach-Colton, M.: The level ancestor problem simplified. *Theoret. Comput. Sci.* **321**(1), 5–12 (2004)
5. Boesch, F., Chen, S., McHugh, J.: On covering the points of a graph with point disjoint paths. In: *Graphs and Combinatorics: Proceedings of the Capital Conference on Graph Theory and Combinatorics at the George Washington University June 18–22, 1973*, pp. 201–212. Springer (1974)
6. Brause, C., Krivoš-Belluš, R.: On a relation between k -path partition and k -path vertex cover. *Discret. Appl. Math.* **223**, 28–38 (2017)
7. Buchsbaum, A., Westbrook, J.: Maintaining hierarchical graph views. In: *Proceedings of the Eleventh Annual ACM-SIAM Symposium on Discrete Algorithms*, pp. 566–575 (2000)
8. Cai, Y., Zhang, X., Qian, J., Sun, Y.: An $O(n)$ Algorithm for ω -Path Partition of Edge-Weighted Forests. Ph.D. thesis (2003)
9. Chen, Y., Goebel, R., Lin, G., Su, B., Xu, Y., Zhang, A.: An improved approximation algorithm for the minimum 3-path partition problem. *J. Comb. Optim.* **38**(1), 150–164 (2019). <https://doi.org/10.1007/s10878-018-00372-z>
10. Goodman, S., Hedetniemi, S.: On the Hamiltonian completion problem. In: *Graphs and Combinatorics: Proceedings of the Capital Conference on Graph Theory and Combinatorics at the George Washington University June 18–22, 1973*, pp. 262–272. Springer (1974)
11. Harel, D., Tarjan, R.: Fast algorithms for finding nearest common ancestors. *SIAM J. Comput.* **13**(2), 338–355 (1984)
12. Hartmanis, J.: Computers and intractability: a guide to the theory of NP-completeness (Michael R. Garey and David S. Johnson). *SIAM Rev.* **24**(1), 90 (1982)
13. Jin, K., Zhang, D., Zhang, C.: Sum-of-max partition under a knapsack constraint. *Comput. Electr. Eng.* **105**, 108521 (2023)
14. Jin, Z., Li, X.: Vertex partitions of r -edge-colored graphs. *Appl. Math.-A J. Chin. Univ.* **23**(1), 120–126 (2008)
15. Klein, P.: Computing the edit-distance between unrooted ordered trees. In: *European Symposium on Algorithms*, pp. 91–102. Springer (1998)
16. Li, S., Yu, W., Liu, Z.: Improved approximation algorithms for the k -path partition problem. *J. Global Optim.* pp. 1–24 (2024)
17. Lu, C., Zhou, Q.: Path covering number and $l(2, 1)$ -labeling number of graphs. *Discret. Appl. Math.* **161**(13–14), 2062–2074 (2013)
18. Luo, R., Zhu, T., Jin, K.: Sum-of-max chain partition of a tree (2025). <https://arxiv.org/abs/2503.11526>
19. Manuel, P.: Revisiting path-type covering and partitioning problems. arXiv preprint [arXiv:1807.10613](https://arxiv.org/abs/1807.10613) (2018)
20. Mehta, D., Sahni, S.: *Handbook of Data Structures and Applications*. Chapman and Hall/CRC, New York (2004)
21. Misra, J., Tarjan, R.: Optimal chain partitions of trees. *Inf. Process. Lett.* **4**(1), 24–26 (1975)
22. Monnot, J., Toulouse, S.: The path partition problem and related problems in bipartite graphs. *Oper. Res. Lett.* **35**(5), 677–684 (2007)
23. Pinar, A., Aykanat, C.: Fast optimal load balancing algorithms for 1D partitioning. *J. Parallel Distrib. Comput.* **64**(8), 974–996 (2004)
24. Slater, P.: Path coverings of the vertices of a tree. *Discret. Math.* **25**(1), 65–74 (1979)

25. Sleator, D., Tarjan, R.: A data structure for dynamic trees. In: Proceedings of the Thirteenth Annual ACM Symposium on Theory of Computing, pp. 114–122 (1981)
26. Vuillemin, J.: A data structure for manipulating priority queues. *Commun. ACM* **21**(4), 309–315 (1978)
27. Yan, J., Chang, G., Hedetniemi, S., Hedetniemi, S.: k -path partitions in trees. *Discret. Appl. Math.* **78**(1–3), 227–233 (1997)